

Python Basics

Data Types & Operations

```
# Numbers
x = 42          # int
y = 3.14        # float
z = x + y       # arithmetic
w = x // 10     # integer division
exp = 2**3      # exponentiation (8)

# Strings
s = "hello"
s[0]            # 'h' (indexing)
s[1:]          # 'ello' (slicing)
s + "world"     # concatenation
len(s)         # 5
```

Control Structures

```
# If statements
if x < 0:
    return "negative"
elif x == 0:
    return "zero"
else:
    return "positive"

# Assertions
assert n >= 1
```

Recursion Basics

Simple Recursion Pattern

Base case + Recursive case

```
def factorial(n):
    if n <= 1:      # Base case
        return 1
    # Recursive case
    return n * factorial(n-1)

# factorial(5) -> 120
```

String Recursion

```
def strlen(s):
    if s == "":     # Base case
        return 0
    return 1 + strlen(s[1:])

# strlen("hello") -> 5
```

Numeric Recursion

```
def logFloor(n):
    # floor(log_10(n))
    assert n >= 1
    if n < 10:
        return 0
    return 1 + logFloor(n/10)

def digitSum(n):
    if n < 10:
        return n
    digit = n // (10**logFloor(n))
    remainder = n - digit*10**logFloor(n)
    return digit + digitSum(remainder)

# digitSum(456) -> 15
```

Linked Lists (LList)

LList Operations

```
from ccomput274 import *

# Create empty list
l = empty()

# Construct list (prepend)
l = cons(1, cons(2, cons(3, empty())))
```

```
# or use shorthand:
l = LL(1, 2, 3) # -> (1, 2, 3)

# Access operations
first(l)        # 1 (head)
rest(l)         # (2, 3) (tail)
isEmpty(l)      # False
```

LList Recursion

```
def doubleList(l):
    if isEmpty(l):
        return l
    return cons(2*first(l),
               doubleList(rest(l)))

# doubleList(LL(1,2,3)) -> (2,4,6)
```

List Building

```
def ascendList(n):
    # Build [0,1,2,...,n]
    if n == 0:
        return cons(0, empty())
    return append(n, ascendList(n-1))

def append(elem, l):
    if isEmpty(l):
        return cons(elem, l)
    return cons(first(l),
               append(elem, rest(l)))
```

List Access

```
def getIth(l, i):
    # Get ith element (0-indexed)
    if i == 0:
        return first(l)
    return getIth(rest(l), i-1)

# myL = LL(5, 10, 27)
# getIth(myL, 1) -> 10
```

Higher-Order Functions

Map (Transform)

```
def map(f, l):
    # Apply f to each element
    if isEmpty(l):
        return empty()
    return cons(f(first(l)),
               map(f, rest(l)))

# map(lambda x: x+3, LL(1,2,3))
# -> (4, 5, 6)
```

Filter (Select)

```
def filter(p, l):
    # Keep elements where p(e) is True
    if isEmpty(l):
        return empty()
    ror = filter(p, rest(l))
    if p(first(l)):
        return cons(first(l), ror)
    return ror

# def isNeg(x): return x < 0
# filter(isNeg, LL(-10,5,-2,0))
# -> (-10, -2)
```

Fold (Reduce)

```
def fold(l, combine, base):
    # Right fold
    if isEmpty(l):
        return base
    return combine(first(l),
               fold(rest(l), combine, base))

# Sum example
def add(x, y): return x + y
# fold(LL(1,2,3), add, 0) -> 6

# Count ones in binary string
def count1(c, n):
    if c == "1":
        return 1 + n
    return n
# foldr("1011", count1, 0) -> 3
```

Build List with Function

```
def buildList(f, n):
    # Apply f to [0..n-1]
    return map(f, ascendList(n-1))

# def pow2(x): return 2**x
# buildList(pow2, 5)
# -> (1, 2, 4, 8, 16)
```

Accumulator Pattern

Tail Recursion (Optimized)

```
@TCO
def ascendListAcc(n, acc):
    if n == 0:
        return cons(0, acc)
    return ascendListAcc(n-1,
                       cons(n, acc))

def ascendList(n):
    return trampoline(
        ascendListAcc(n, empty()))
```

Benefits: Space-efficient, avoids stack overflow for large inputs.

Finding Extremes

```
def exComb(n, pair):
    # Update (MIN, MAX)
    if n < first(pair):
```

```
        return cons(n, rest(pair))
    if n > first(rest(pair)):
        return LL(first(pair), n)
    return pair

def extremes(l):
    return foldl(1, exComb,
                LL(first(l), first(1)))

# extremes(LL(5,2,9,1)) -> (1,9)
```

String Manipulation

Pig Latin

```
def pl(s):
    vowels = "aeiou"
    if s[0] in vowels:
        return s + "way"
    else:
        return s[1:] + s[0] + "ay"

# pl("hello") -> "ellohay"
# pl("orange") -> "orangeay"
```

Leet Speak

```
def convertChar(c):
    conversions = {
        'a':'4', 'A':'4',
        'l':'1', 'L':'1',
        't':'7', 'T':'7',
        'e':'3', 'E':'3',
        's':'5', 'S':'5',
        'o':'0', 'O':'0'
    }
    return conversions.get(c, c)

def leetSpeak(s):
    if s == "":
        return ""
    if s[:2].lower() == "er":
        return "z0r" + leetSpeak(s[2:])
    return convertChar(s[0]) + \
        leetSpeak(s[1:])
```

String Equality

```
def streq(s1, s2):
    if s1 == "" and s2 == "":
        return True
    if s1 == "" or s2 == "":
        return False
    if s1[0] != s2[0]:
        return False
    return streq(s1[1:], s2[1:])
```

Common Patterns

List Operations

```
def sum(lon):
    if isEmpty(lon):
        return 0
    return first(lon) + sum(rest(lon))

def length(lon):
    if isEmpty(lon):
        return 0
    return 1 + length(rest(lon))

def mean(lon):
    return sum(lon) / length(lon)
```

Parity Check

```
def parity(s):
    # True if even number of 1s
    def count1(c, n):
        return (1+n) if c=="1" else n
    return foldr(s, count1, 0) % 2 == 0

# Alternative with boolean
def checkOne(c, b):
    return not b if c=="1" else b
def parity2(s):
    return foldr(s, checkOne, True)
```

Efficiency Tips

Time Complexity

- **Linear recursion:** $O(n)$ time

- **Tree recursion:** Can be $O(2^n)$
- **List traversal:** $O(n)$
- **Append to end:** $O(n)$ (prepend is $O(1)$)

Space Complexity

- **Regular recursion:** $O(n)$ stack space
- **Tail recursion + TCO:** $O(1)$ stack
- **Accumulator pattern:** More efficient

Fast vs Slow

```
# SLOW: Append to end each time
def ascendListSlow(n):
    if n == 0:
        return cons(0, empty())
    return append(n, ascendListSlow(n-1))
#  $O(n^2)$  - appending is  $O(n)$ 

# FAST: Use accumulator
def ascendListFast(n, acc=empty()):
    if n == 0:
        return cons(0, acc)
    return ascendListFast(n-1, cons(n, acc))
#  $O(n)$  - prepending is  $O(1)$ 
```

Key Concepts

Functional Programming

- **Immutability:** Don't modify data
- **First-class functions:** Pass functions as arguments
- **Pure functions:** No side effects
- **Recursion:** Instead of loops

Design Recipe

1. **Signature:** Input/output types
2. **Purpose:** What it does
3. **Examples:** Test cases
4. **Body:** Implementation
5. **Test:** Verify examples

Recursion Template

```
def recursive_fn(data):
    # 1. Base case(s)
    if base_condition:
        return base_value

    # 2. Recursive case
    # - Make problem smaller
    # - Recurse on smaller problem
    ror = recursive_fn(smaller_data)

    # 3. Combine results
    return combine(current, ror)
```